Department of Electronic and Telecommunication Engineering

University of Moratuwa, Sri Lanka



Battery-Aware Return-to-Home for a Delivery Drone Using an Extended Kalman Filter

Basnayake B.M.N.S

Index No.: 220072R

EN4594-Autonomous Systems

Submitted in partial fulfillment of the requirements for course module EN4594

October 20, 2026

Abstract

A delivery drone must decide when to abandon its route and fly home, yet it cannot measure its battery's remaining charge directly: its current sensor is noisy and biased, and its voltage sags under load. This report develops an Extended Kalman Filter (EKF) that estimates the state of charge (SoC) and its uncertainty from these two signals, and a risk-aware return-to-home rule that turns back when the probability of landing with a 10% reserve falls below 95%. Both are tested in a MATLAB simulator whose battery deliberately differs from the filter's equivalent-circuit model. A design-stage investigation showed that resistance mismatch dominated the SoC error, so the filter learns the cell resistance during flight; this reduced the SoC RMSE on a nominal flight to 0.97%, against 4.36% for charge counting. A five-state filter that also estimates sensor bias and capacity kept honest uncertainty bands in every nominal, biased and aged-battery scenario and was chosen for decisions. In 500 random missions, the risk-aware rule had no battery failures and broke the reserve in 2.8% of missions, against 38.0% for the same filter without its uncertainty and 30.4% battery failures for charge counting. Its breach rate stayed within the promised level at every threshold tested, showing that its probabilities were calibrated.

Contents

Abstract **i**

1 Introduction **1**

2 Background **3**

3 Methodology **7**

4 Implementation **14**

5 Results **17**

6 Discussion **24**

7 Conclusion **27**

References **28**

A MATLAB Code **30**

Chapter 1

Introduction

1.1 Problem and motivation

A delivery drone must decide, during flight, when to stop delivering and return home. It cannot measure its remaining charge, the *state of charge* (SoC), directly. It only has a noisy, slightly biased current sensor and a voltage sensor whose reading dips under heavy load, so a low voltage does not always mean an empty battery.

The decision can fail in two ways. Turning back too late risks losing the drone and its payload. Turning back too early wastes battery and deliveries. A fixed safety margin trades one failure for the other. A better decision needs both an SoC estimate and a measure of how uncertain it is, so that the margin can shrink when the estimate is precise and grow when it is not.

1.2 Why an Extended Kalman Filter

A Kalman filter combines a model-based prediction with a measurement, weights each by how much it can be trusted, and reports its own uncertainty. The standard filter assumes linear equations, but a battery's resting voltage is a curved function of its SoC (Section 2.1). The Extended Kalman Filter (EKF) handles this by using the tangent of the curve at the current estimate. The curve is smooth and only mildly bent, so the EKF fits well, and it is the classic approach to SoC estimation in battery management systems [1, 2]. A particle filter would also work, but at a much higher computational cost for little benefit here [3].

1.3 Objectives

1. Build a MATLAB simulator of a delivery drone and its battery that gives the true SoC and realistic, imperfect sensor readings.

2. Design an EKF that estimates the SoC and its uncertainty.

3. Design a return-to-home (RTH) rule that uses both the estimate and its uncertainty.
4. Compare the EKF and the RTH rule with simpler methods (charge counting and a voltage threshold) in single flights and over many random missions.

1.4 Contributions

- An EKF that learns the battery's internal resistance during flight. Tests showed that a wrong resistance value, not the faulty current sensor, caused most of the SoC error (Section 3.4.5).
- A choice of the filter used for decisions based on how honest its uncertainty is, not only on its accuracy. The most accurate filter was confidently wrong on an aged battery (Section 3.4.5).
- An RTH rule that turns back when the probability of getting home with a safety reserve drops below 95%, so the margin grows automatically when the filter is unsure (Section 3.5).
- A safety-versus-wasted-battery trade-off that shows what each extra level of safety costs (Section 5.5).

Chapter 2

Background

2.1 Battery basics

The *capacity* C_n is the charge a full battery can deliver, in ampere-hours (Ah); it falls as the battery ages. The *state of charge* (SoC) is the fraction of the capacity that remains, from 1 (full) to 0 (empty).

When a battery rests, its voltage depends only on its SoC. This resting voltage is the *open-circuit voltage* (OCV). The OCV curve (Figure 2.1) is curved, so a straight-line model cannot describe it, and its slope changes: about 25 mV per 1% SoC near empty, but only 5 mV per 1% between 20% and 40%. Where the curve is flat, the voltage says little about the SoC, so a good filter should become less certain there.

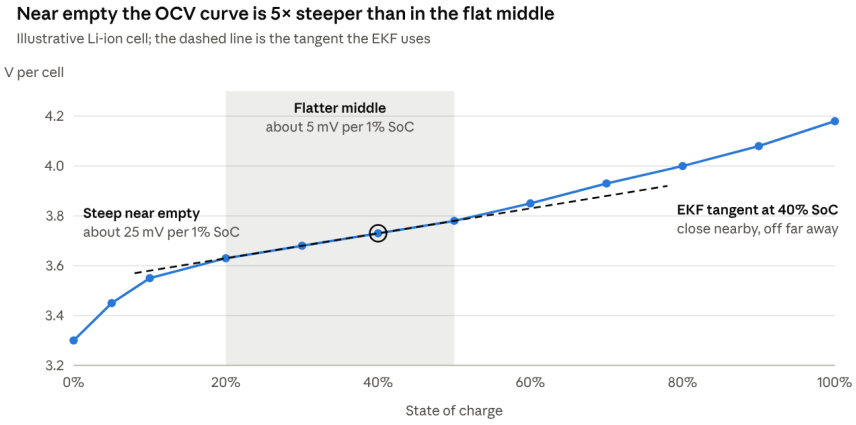


Figure 2.1: OCV curve of one cell. It is steep near empty and flat between about 20% and 50% SoC. The dashed tangent at 40% SoC is what the EKF uses: close to the curve nearby, wrong far away.

Under load the voltage drops in two ways. An instant drop $R_0 I$ comes from the internal resistance R_0, where I is the current (positive for discharge):

$$\Delta V_{R_0} = R_0\, I. \tag{2.1}$$

At 30 A take-off current and $R_0 = 8\,\mathrm{m\Omega}$ this is 0.24 V per cell, so the battery looks emptier than it is. A slower drop V_1 builds up over tens of seconds, modelled by a resistor R_1 in parallel with a capacitor C_1:

$$C_1 \frac{dV_1}{dt} = I - \frac{V_1}{R_1}, \tag{2.2}$$

with time constant $\tau_1 = R_1 C_1$ (30 s here). Together these form the equivalent circuit of Figure 2.2. The sensor reads

$$V = \mathrm{OCV}(\mathrm{SoC}) - R_0\, I - V_1. \tag{2.3}$$

Only V and I are measured; the SoC and V_1 must be estimated.

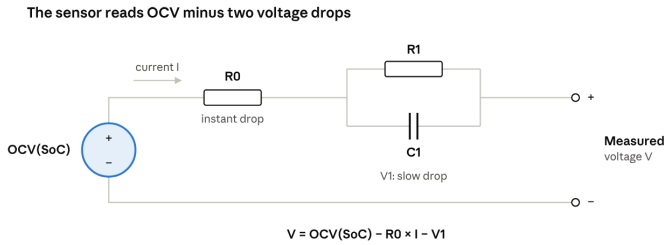


Figure 2.2: One-RC equivalent circuit used by the EKF. The simulated "true" battery adds a second, slower RC pair (Section 3.2).

The simplest SoC tracker is *charge counting*: start from a known SoC and subtract the charge drawn. With time step Δt (s) and current I_k (A),

$$\mathrm{SoC}_k = \mathrm{SoC}_{k-1} - \frac{I_k\, \Delta t}{3600\, C_n}. \tag{2.4}$$

It never corrects itself, so a wrong start stays wrong, a wrong capacity makes it drift, and a current-sensor bias b adds an error that grows with time:

$$e(t) = -\frac{b\, t}{3600\, C_n}. \tag{2.5}$$

A 0.5 A bias over 15 minutes is 2.5% of a 5 Ah battery. Charge counting is used as a baseline.

2.2 From the Kalman filter to the Extended Kalman Filter

A Kalman filter balances a prediction against a measurement. For the drone, charge counting (2.4) is the prediction and the voltage (2.3) is the measurement. The hidden quantities form the state x_k; the measured current is the input u_k and the measured voltage is the measurement z_k:

$$x_k = f(x_{k-1}, u_k) + w_k, \qquad z_k = h(x_k, u_k) + v_k, \tag{2.6}$$

where w_k and v_k are zero-mean, bell-shaped (Gaussian) errors with covariances Q (model error) and R (sensor error). The filter keeps an estimate $\hat{x}_k$ and its uncertainty (covariance) P_k.

For a linear model, $f = Fx + Bu$ and $h = Hx$, each step has two parts. **Predict** with the model (uncertainty grows), then **update** with the measurement (uncertainty shrinks):

$$\hat{x}_k^- = F\hat{x}_{k-1} + Bu_k, \qquad P_k^- = FP_{k-1}F^\top + Q, \tag{2.7}$$

$$\tilde{y}_k = z_k - H\hat{x}_k^-, \qquad S_k = HP_k^-H^\top + R, \tag{2.8}$$

$$K_k = P_k^-H^\top S_k^{-1}, \qquad \hat{x}_k = \hat{x}_k^- + K_k\tilde{y}_k, \tag{2.9}$$

$$P_k = (\mathbb{I} - K_kH)P_k^-. \tag{2.10}$$

Here "$-$" means before the measurement, $\tilde{y}_k$ is the *innovation* (the filter's surprise), S_k its expected variance, and K_k the *Kalman gain*. The gain is large when the prediction is uncertain and small when the sensor is noisy or the measurement says little about the state, as on the flat part of the OCV curve. Q and R are the tuning knobs: a large Q makes the filter fast but jumpy, a large R smooth but slow.

The battery model is not linear, because of OCV(SoC). The EKF replaces f and h, at every step, by their tangent at the current estimate. The slopes form the *Jacobians*

$$F_k = \left.\frac{\partial f}{\partial x}\right|_{\hat{x}_{k-1},\,u_k}, \qquad H_k = \left.\frac{\partial h}{\partial x}\right|_{\hat{x}_k^-,\,u_k}, \tag{2.11}$$

which replace F and H in (2.7)–(2.10), while the state and expected measurement use the full nonlinear f and h. For the battery the key entry of H_k is the OCV slope at the current SoC estimate, the tangent in Figure 2.1. This works well when the curve is smooth and the uncertainty small, as here [3, 4].

2.3 A worked example

One EKF step with SoC as the only state (`learning/day1_ekf_by_hand.m`) shows the gain at work. The filter starts at $80\% \pm 5\%$. Drawing 20 A for 10 s predicts 78.89%. The model then expects 3.832 V, but the sensor reads 3.80 V, a surprise of -32 mV. With an OCV slope of 7 mV per 1% SoC and a 10 mV sensor, the gain is $K = 1.32\,\mathrm{V}^{-1}$, and the estimate moves to $74.6\% \pm 1.4\%$: one voltage reading cut the uncertainty from 5% to 1.4%. With a noisy 50 mV sensor, or a confident $\pm 1\%$ start, the gain falls to about 0.47 and the estimate moves much less.

2.4 Related work

Plett's three-part paper [1, 2, 5] is the classic reference for EKF battery estimation, including adding unknown parameters such as resistance or capacity to the state, as done here; his textbook [6] covers equivalent-circuit models in depth. MathWorks provides a MATLAB EKF battery example [7]. Simon [3] covers the KF, EKF, unscented Kalman filter (UKF) and particle filter, and Welch and Bishop [4] give a short introduction.

For drones, an EKF has been used for UAV battery SoC estimation [8], and prediction of remaining flying time, the quantity an RTH decision depends on, has been verified for electric unmanned vehicles [9]. Dibbern et al. [10] compared an EKF, a UKF and charge counting on delivery and inspection missions, and found the UKF more accurate and parameter identification more important than the load profile. Public flight and cell datasets [11–14] are available for future validation; this project uses simulation so that the true SoC is always known.

Chapter 3

Methodology

3.1 System overview

Figure 3.1 shows the system. The simulator flies the drone and computes the true battery state. The sensors turn the true current and voltage into imperfect readings. The EKF and the two baselines (charge counting and a voltage threshold) see only these readings, and each feeds a turn-back decision. Only the simulator knows the truth, so every estimate and every decision can be scored exactly.

The drone carries a six-cell series lithium-ion pack of 5 Ah. The same current flows through every cell, so one cell is modelled; all voltages and battery parameters are per cell. Everything runs once per second ($\Delta t = 1\,\mathrm{s}$); I_k is the current during step k and every state with index k is its value at the end of that step.

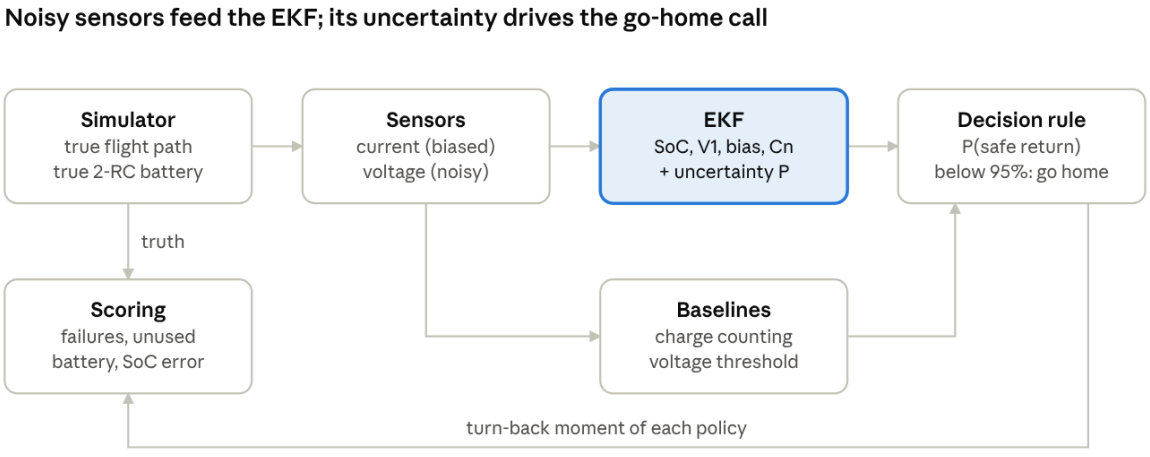


Figure 3.1: System overview. The EKF and the baselines see only the noisy sensor readings; the simulator's truth is used only for scoring. (The EKF block lists four states; the final filter also learns R_0, Section 3.4.4.)

3.2 Battery models: truth versus filter

A filter tested against its own model always looks good, and a real filter never has the exact model. The simulated battery is therefore deliberately different from the filter's model (Table 3.1).

With the current held constant during a step, an RC pair (2.2) updates exactly as

$$V_{i,k} = a_i\, V_{i,k-1} + R_i\,(1 - a_i)\, I_k, \qquad a_i = e^{-\Delta t/(R_i C_i)}. \tag{3.1}$$

The true battery has a second, slow RC pair (R_2, C_2) that the filter does not know about, and an OCV curve shifted by δ_{OCV}:

$$\mathrm{SoC}_k = \mathrm{SoC}_{k-1} - \frac{I_k\,\Delta t}{3600\,C_n}, \qquad V_{1,k},\, V_{2,k} \text{ from (3.1)}, \tag{3.2}$$

$$V_k = \mathrm{OCV}(\mathrm{SoC}_k) + \delta_{\mathrm{OCV}} - R_0\, I_k - V_{1,k} - V_{2,k}. \tag{3.3}$$

The battery is dead if $V_k < 3.0\,\mathrm{V}$ while current flows.

Both models use the same representative lithium-ion OCV table (Figure 2.1), smoothed with shape-preserving cubic interpolation (`pchip`) so its slope does not jump. The EKF needs the slope, found by a central difference over ±0.5% SoC (one-sided at the ends of the table):

$$\frac{d\,\mathrm{OCV}}{d\,\mathrm{SoC}}(s) \approx \frac{\mathrm{OCV}(s^+) - \mathrm{OCV}(s^-)}{s^+ - s^-}, \qquad s^{\pm} = s \pm 0.005. \tag{3.4}$$

Table 3.1: Battery parameters per cell (`config/battery_params.m`).

Parameter	Filter	Truth	Note
Circuit	1 RC pair	2 RC pairs	
C_n	5.0 Ah	5.0 Ah	4.25 Ah in E4
R_0	8 mΩ	8.8 mΩ	+10%
R_1, C_1	6 mΩ, 5000 F	5.4 mΩ, 5500 F	$\tau_1 \approx 30$ s
R_2, C_2	–	1.5 mΩ, 266 667 F	$\tau_2 = 400$ s; 4 mΩ in E6
δ_{OCV}	0	−5 mV	

The slow pair is the most visible difference: with $\tau_2 = 400\,\mathrm{s}$ its voltage keeps building through most of a 15-minute flight, and the filter has no state for it. A first setting, $R_2 = 4\,\mathrm{m\Omega}$, was used for the simulator test (Section 5.1); at 15 A it adds about 63 mV of unmodelled drop, which on the flat part of the curve looks like up to 13% SoC error. That is harsher than a reasonably identified model, so the default was reduced to 1.5 mΩ (about 23 mV) and the strong setting kept as stress test E6.

3.3 Sensors and mission

The sensors read

$$I_{m,k} = I_k + b + w_{I,k}, \qquad V_{m,k} = V_k + v_k, \tag{3.5}$$

with a current bias b (0.5 A by default), current noise $\sigma_I = 0.2\,\mathrm{A}$ and voltage noise $\sigma_V = 10\,\mathrm{mV}$.

The drone takes off (30 s at 30 A), cruises at 10 m/s to each drop-off (15 A $\times$ $(1 + w)$, where w is a wind factor), hovers 30 s at 20 A to deliver, flies home and lands (40 s at 18 A). A slowly varying random load is added while flying:

$$\varepsilon_k = 0.9\,\varepsilon_{k-1} + 0.3\,n_k, \quad n_k \sim \mathcal{N}(0,1), \qquad I_k = \max(I_{\mathrm{demand},k} + \varepsilon_k,\, 0), \tag{3.6}$$

which wanders by about ± 0.7 A. The *short* route has three drop-offs and ends with charge to spare; the *long* route has six drop-offs reaching 9 km and is too far for one battery, so the return-to-home rule must act.

3.4 The Extended Kalman Filter

3.4.1 Model

The state, input and measurement are

$$x_k = \begin{bmatrix} \mathrm{SoC}_k & V_{1,k} & b_k & C_{n,k} & R_{0,k} \end{bmatrix}^\top, \qquad u_k = I_{m,k}, \qquad z_k = V_{m,k}. \tag{3.7}$$

The filter's best guess of the true current is $I_{m,k} - b$. The process model is charge counting and the RC update with this corrected current, while the bias, capacity and resistance are treated as constants that may wander slowly (this is how a filter learns an unknown constant):

$$\mathrm{SoC}_k = \mathrm{SoC}_{k-1} - \frac{(I_{m,k} - b_{k-1})\,\Delta t}{3600\,C_{n,k-1}}, \tag{3.8}$$

$$V_{1,k} = a\,V_{1,k-1} + R_1(1-a)(I_{m,k} - b_{k-1}), \tag{3.9}$$

$$b_k = b_{k-1} + w_b, \qquad C_{n,k} = C_{n,k-1} + w_C, \qquad R_{0,k} = R_{0,k-1} + w_R. \tag{3.10}$$

The measurement model is the circuit equation (2.3):

$$V_{m,k} = \mathrm{OCV}(\mathrm{SoC}_k) - V_{1,k} - R_{0,k}\,(I_{m,k} - b_k) + v_k. \tag{3.11}$$

3.4.2 Jacobians

Differentiating (3.8)–(3.11) with respect to the state gives

$$F_k = \begin{bmatrix} 1 & 0 & \frac{\Delta t}{3600\,\hat{C}_n} & \frac{(I_m - \hat{b})\,\Delta t}{3600\,\hat{C}_n^2} & 0 \\ 0 & a & -R_1(1-a) & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \qquad H_k^\top = \begin{bmatrix} d\,\mathrm{OCV}/d\,\mathrm{SoC} \\ -1 \\ \hat{R}_0 \\ 0 \\ -(I_m - \hat{b}) \end{bmatrix}. \tag{3.12}$$

The OCV slope makes the voltage correct the SoC strongly near empty and weakly in the flat middle. The resistance entry grows with the current, so the resistance shows up most when the current changes. The capacity entry of H is zero: capacity is learned only slowly, through how the SoC drifts.

3.4.3 Noise

The current-sensor noise enters the prediction through

$$G_k = \frac{\partial f}{\partial I_m} = \begin{bmatrix} -\frac{\Delta t}{3600\,\hat{C}_n} & R_1(1-a) & 0 & 0 & 0 \end{bmatrix}^{\top}.$$

Model errors without a single physical source are covered by small diagonal terms:

$$Q_k = G_k\,\sigma_I^2\,G_k^{\top} + \operatorname{diag}(q_{\mathrm{SoC}},\, q_{V_1},\, q_b,\, q_{C_n},\, q_{R_0})\,, \qquad R = \sigma_V^2 + (R_0\,\sigma_I)^2 + \sigma_{\mathrm{model}}^2. \quad (3.13)$$

In R, the second term is the current noise seen through the resistance, and $\sigma_{\mathrm{model}} = 10\,\mathrm{mV}$ is an allowance for the filter's imperfect model. The tuning values are in Table 3.2.

Table 3.2: Filter tuning (`estimators/ekf_init.m`).

	q_{SoC}	q_{V_1} (V^2)	q_b (A^2)	q_{C_n} (Ah^2)	q_{R_0} (Ω^2)	R (V^2)
Per step	10^{-8}	10^{-6}	10^{-6}	10^{-7}	10^{-9}	$\approx 2.0 \times 10^{-4}$

The starting SoC is read from the resting voltage before take-off (the average of 10 readings in E3–E8 and the Monte Carlo study) with ±3% uncertainty; the other states start at $V_1 = 0$, $b = 0$, $C_n = 5\,\mathrm{Ah}$ and $R_0 = 8\,\mathrm{m}\Omega$:

$$\hat{x}_0 = \begin{bmatrix} \widehat{\mathrm{SoC}}_0 & 0 & 0 & 5.0 & 0.008 \end{bmatrix}^{\top}, \quad P_0 = \operatorname{diag}\!\left(0.03^2,\, 0.01^2,\, 1^2,\, 0.5^2,\, 0.004^2\right). \quad (3.14)$$

The covariance is updated in a numerically stable (Joseph) form that keeps it symmetric and positive [3], and the SoC, capacity and resistance are clamped to physical ranges:

$$P_k = (\mathbb{I} - K_k H_k)\,P_k^{-}\,(\mathbb{I} - K_k H_k)^{\top} + K_k R K_k^{\top}. \quad (3.15)$$

Whether the filter's uncertainty is honest is checked in three ways: the true error should stay inside $\pm 3\sigma$ about 99.7% of the time; the normalised innovation squared (NIS, $\tilde{y}^2/S$) should average about 1; and the normalised SoC error squared (NEES, $e_{\mathrm{SoC}}^2/P_{11}$, possible only in simulation) should average about 1.

3.4.4 Filter stages

Three versions of the filter are compared (Table 3.3). A state that a stage does not learn is frozen at its nominal value with zero uncertainty, which is the same as leaving it out.

Table 3.3: Filter stages.

Stage	Learned states	Role
A	SoC, V_1	Basic EKF
B	SoC, V_1, R_0	Most accurate on a healthy battery
C	all five	Used for the return-to-home decision

3.4.5 Why the resistance became a state

The original plan was a four-state filter without R_0. Early tests in a Python copy of the simulator showed that stage A read the SoC about 2.5% too low and was overconfident: its $\pm 3\sigma$ band held the truth only about 16% of the time. Switching on one model mismatch at a time (Table 3.4) found the cause: a wrong resistance. The true battery sags more than the filter expects, and the filter can only explain the extra drop as a lower SoC; at 15 A, a 10% resistance error is 12 mV, which on the flat part of the curve looks like 2.4% less charge.

Table 3.4: Early test (Python copy, E1): SoC error of stage A with one mismatch at a time.

Mismatch switched on	SoC RMSE
None (perfect model)	0.12%
R_0 10% higher	1.85%
Slow RC pair	1.62%
OCV 5 mV lower	0.73%
Current bias 0.5 A	0.43%

Telling the filter its sensor was noisier (a larger R) did not help (2.49%), because the error is systematic, not random. Learning R_0 did: a drop that appears only under load must be resistance, not lower SoC. This gave stage B. The allowance $\sigma_{\text{model}} = 10\,\text{mV}$ was then chosen because it kept stage B's band honest (98% coverage over 20 seeds, against 90% without it).

Learning the bias and capacity as well (stage C) cannot separate them within one flight. Small errors in b and R_0 change the voltage by

$$\delta V \approx \hat{R}_0\,\delta b - (I_m - \hat{b})\,\delta R_0, \tag{3.16}$$

which cancel at a steady current, and the capacity shows only through slow SoC drift. What the voltage does pin down is the drain rate

$$r = \frac{I_m - b}{C_n}, \tag{3.17}$$

so stage C's SoC stays accurate even when its b and C_n are individually wrong. On an aged battery, stage B blames the lower voltage on its resistance and overestimates the SoC, which is the dangerous direction, while stage C stays honest (Section 5.2.3). Stage C therefore drives the return-to-home decision.

3.5 Risk-aware return-to-home rule

Every second, the drone estimates the charge needed to fly home in a straight line at $v_{\text{cruise}} = 10\,\text{m/s}$ and land:

$$t_{\text{cruise},k} = \frac{d_{\text{home},k}}{v_{\text{cruise}}}, \qquad \Delta\text{SoC}_{\text{home},k} = \frac{\left(\hat{I}_{\text{cr},k} - \hat{b}_k\right) t_{\text{cruise},k} + I_{\text{land}}\, t_{\text{land}}}{3600\,\hat{C}_{n,k}}. \tag{3.18}$$

The cruise term uses the filter's own drain rate (3.17), the quantity the voltage makes reliable. $\hat{I}_{\mathrm{cr}}$ is the cruise current as measured, learned in flight with a 50-second moving average:

$$\hat{I}_{\mathrm{cr},k} = 0.98\,\hat{I}_{\mathrm{cr},k-1} + 0.02\,I_{m,k}, \qquad \hat{I}_{\mathrm{cr},0} = 15\,\mathrm{A}, \tag{3.19}$$

and the landing uses its nominal current ($I_{\mathrm{land}} = 18\,\mathrm{A}$, $t_{\mathrm{land}} = 40\,\mathrm{s}$).

The safety margin after the trip and a reserve of $\mathrm{SoC}_{\mathrm{res}} = 10\%$ is converted into a probability:

$$m_k = \widehat{\mathrm{SoC}}_k - \Delta\mathrm{SoC}_{\mathrm{home},k} - \mathrm{SoC}_{\mathrm{res}}, \tag{3.20}$$

$$\sigma_{\mathrm{total},k} = \sqrt{P_{k,11} + \left(0.1\,\Delta\mathrm{SoC}_{\mathrm{home},k}\right)^2}, \tag{3.21}$$

$$P_{\mathrm{safe},k} = \Phi\left(\frac{m_k}{\sigma_{\mathrm{total},k}}\right). \tag{3.22}$$

The first uncertainty term comes from the EKF; the second allows 10% of the trip cost for wind and load surprises. Φ is the standard normal distribution function. The drone turns back when $P_{\mathrm{safe}} < 95\%$, that is, when the margin falls below 1.645 standard deviations. When the filter is confident the buffer is small and the drone flies longer; when it is unsure the buffer grows automatically. If the uncertainty is honest, no more than 5% of missions should break the reserve, which the Monte Carlo study tests directly.

Four policies are compared on the same missions (Table 3.5). They decide only while the drone is cruising or hovering; once triggered, the drone flies straight home.

Table 3.5: Return-to-home policies.

Policy	Turn back when
A. Voltage threshold	measured voltage stays below 3.5 V for 5 s
B. Charge counting	charge-counted SoC minus trip cost and reserve < 0 (rated capacity, no bias correction)
C. EKF estimate only	$m_k < 0$
D. EKF risk-aware	$P_{\mathrm{safe},k} < 95\%$

Each mission is scored on whether the battery **died**, whether the **reserve was broken** (died or landed below 10%), the **unused battery** at landing above the reserve, and the number of **deliveries**.

3.6 Experiments

Eight experiments on the short route test the filter (Table 3.6); single runs use one fixed random seed and E8 repeats five scenarios over 20 seeds. The four policies are then compared on 500 random long-route missions (Table 3.7), each policy flying exactly the same conditions and sensor noise, and policy D's threshold (80–99.9%) and policy A's voltage (3.40–3.60 V) are swept on 200 of them. A single demonstration mission (aged battery, sensor reading 0.5 A low, 10% headwind) shows how each policy decides.

Table 3.6: Estimation experiments.

#	Experiment	Set-up
E1	Nominal flight	True start 95%, read from the resting voltage
E2	Wrong start	True 85%, estimators assume 100% (±20%)
E3	Sensor bias	Bias 0.8 A; stages B and C
E4	Aged battery	True capacity 4.25 Ah, start 100%; stages B and C
E5	Tuning	Stage B with Q or R scaled by 0.01 or 100
E6	Strong mismatch	Slow pair $R_2 = 4\,\mathrm{m\Omega}$; stages A, B, C
E7	Flat OCV region	σ_{SoC} against true SoC, stages B and C
E8	Stage comparison	20 seeds × E1, E2, E3, E4, E6 × stages A, B, C

Table 3.7: Random conditions for each Monte Carlo mission, drawn uniformly.

Factor	Range
Starting SoC	60–100%
Current-sensor bias	−1 to +1 A
Wind (cruise current)	−20 to +20%
Battery capacity	80–100% of 5 Ah
True R_0, R_1, C_1, R_2	each ±15%

Chapter 4

Implementation

4.1 Tools and code structure

The project uses base MATLAB only, with a time step of 1 s. Every run fixes the random seed, so it can be repeated exactly, and runs that are compared use the same seed and therefore the same sensor noise. The code is split into short files that each do one job (Table 4.1); the core files are listed in Appendix A.

Table 4.1: Code organisation in `drone_battery_ekf/`.

Folder	What it does
`config/`	Battery and sensor parameters; OCV curve and its slope; SoC from resting voltage
`sim/`	True 2-RC battery, sensors, drone flight phases, routes, one complete flight, random Monte Carlo conditions, performance measures
`estimators/`	EKF set-up (stages A–C) and one predict–update step; charge counting
`decision/`	Trip cost, probability of a safe return, the four policies
main scripts	Simulator test, E1–E2, E3–E8, return-to-home study (demo, Monte Carlo, sweep)

4.2 The EKF in code

Listing 4.1 is the heart of `ekf_step.m`. Lines 16–20 are the process model (3.8)–(3.10), lines 22–30 the Jacobian F and the noise Q of (3.12) and (3.13), lines 33–39 the expected voltage (3.11), the innovation and H, and lines 42–46 the gain and the stable covariance update (3.15). Because there is one voltage measurement, S is a single number and no matrix inverse is needed. All five states are always present; a stage freezes the states it does not learn by giving them zero uncertainty.

```
15 % ---------- 1. Predict: move the estimate with the model ----------
16 x_pred = [soc - I*dt/(3600*Cn);              % charge counting
```

```
          a*v1 + f.R1*(1 - a)*I;              % slow voltage relaxes toward R1*I
          b;                                  % bias, capacity and resistance
          Cn;                                 %   stay the same (apart from
          R0];                                %   slow random wander)

F = eye(5);                                   % Jacobian of the process model
F(1,3) = dt/(3600*Cn);                        %   dSoC/db
F(1,4) = I*dt/(3600*Cn^2);                    %   dSoC/dCn
F(2,2) = a;                                   %   dV1/dV1
F(2,3) = -f.R1*(1 - a);                       %   dV1/db

G = [-dt/(3600*Cn); f.R1*(1 - a); 0; 0; 0];   % how current-sensor noise enters
Q = G * f.sigI^2 * G' + diag(f.q);
P_pred = F * f.P * F' + Q;                    % uncertainty grows

% ---------- 2. Compare: expected vs measured voltage ----------
[ocv, docv] = ocv_lookup(x_pred(1), f.ocv_offset);
I_pred = Im - x_pred(3);
V_pred = ocv - x_pred(2) - x_pred(5)*I_pred;
y = Vm - V_pred;                              % innovation (the surprise)

H = [docv, -1, x_pred(5), 0, -I_pred];        % Jacobian of the measurement model
S = H * P_pred * H' + f.Rv;                   % expected size of the surprise (one
    number)

% ---------- 3. Update: move by K x surprise ----------
K = P_pred * H' / S;                          % Kalman gain (5x1)
x = x_pred + K * y;
IKH = eye(5) - K*H;
P = IKH * P_pred * IKH' + K * f.Rv * K';      % Joseph form: stays symmetric and
   positive
P = (P + P') / 2;
```

Listing 4.1: Predict and update in `estimators/ekf_step.m`.

4.3 The decision rule in code

Listing 4.2 turns the filter's estimate into the probability of a safe return, following (3.20)–(3.22). The normal distribution is written with `erfc`, so no toolbox is needed.

```
soc = f.x(1);  b = f.x(3);  Cn = f.x(4);
dsoc   = trip_soc(dist_home, I_cruise, b, Cn, m);
margin = soc - dsoc - rp.reserve;

sig_soc   = sqrt(f.P(1,1));          % from the EKF: how unsure is the SoC
sig_trip  = rp.trip_frac * dsoc;     % wind and load surprises on the way home
sig_total = sqrt(sig_soc^2 + sig_trip^2);

ps = 0.5 * erfc(-margin / (sqrt(2) * sig_total));   % normal CDF, base MATLAB
```

Listing 4.2: Probability of a safe return in `decision/p_safe_return.m`.

4.4 Running the experiments

Every second, a flight runs the same loop: the drone model gives the current demand, the true battery responds, the sensors add bias and noise, the EKF and charge counting update, and, in return-to-home missions, the policy decides whether to turn back. `run_flight.m` flies the experiments E1–E8 and `run_mission_rth.m` the return-to-home missions; both record the truth and the estimates for scoring. Four main scripts reproduce every result: `main_day2_simulator.m` (simulator test), `main_day3_ekf.m` (E1–E2), `main_day4_experiments.m` (E3–E8, about 20 s for the 300 flights of E8) and `main_day5_rth.m` (demonstration, 500-mission Monte Carlo in about 95 s, and threshold sweep).

Chapter 5

Results

5.1 Simulator check

The short mission, flown once without a filter, took 14.2 min and used the battery from 95% to 18.5%. Figure 5.1 shows the voltage sag that fools simple gauges: the measured voltage sits up to 0.37 V below the resting voltage, jumps up when take-off ends and steps down at every hover, while the true SoC falls smoothly. Charge counting with the 0.5 A sensor bias ended 2.3% low, as (2.5) predicts (2.4%). This run used the first, stronger slow RC pair ($R_2 = 4\,\mathrm{m\Omega}$), whose widening gap led to the milder default (Section 3.2).

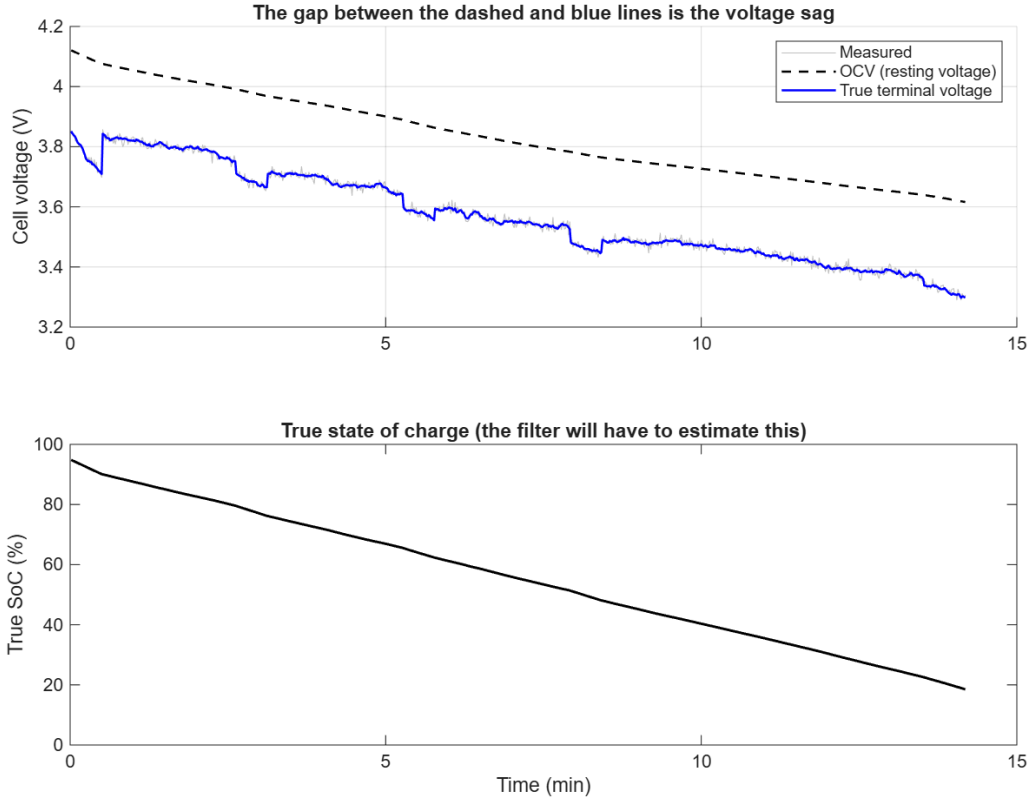


Figure 5.1: Simulator check. Top: measured and true terminal voltage against the resting voltage (OCV); the gap is the sag. Bottom: true SoC.

5.2 Estimation experiments

5.2.1 Normal flight and wrong start (E1, E2)

Table 5.1 compares the three estimators on the same sensor readings. Stage B, which learns the resistance, was the most accurate in both flights and its band always contained the truth. Stage A was biased low and overconfident, exactly as the early tests predicted (Section 3.4.5).

Table 5.1: SoC estimation in E1 and E2. Final: error at landing. In $\pm 3\sigma$: how often the truth was inside the filter's band. $< 2\%$ after: time after which the error stayed below 2%.

	Estimator	RMSE	Final	In $\pm 3\sigma$	NIS	$< 2\%$ after
E1	Charge counting	4.36%	−5.50%	–	–	–
	EKF stage A	2.51%	−3.89%	17%	0.54	never
	EKF stage B	0.97%	−1.53%	100%	0.51	30 s
E2	Charge counting	13.85%	+12.64%	–	–	–
	EKF stage A	2.49%	−2.49%	18%	0.60	never
	EKF stage B	1.32%	−0.38%	100%	0.51	42 s

In E2 (Figure 5.2) the battery really started at 85% but every estimator assumed full. Charge counting never noticed: at landing the battery held about 9%, while charge counting believed about 21%, which on a longer flight would mean running out while thinking a fifth of the battery was left. Stage B corrected the wrong start within 42 s. Its key moment, in both flights, was the end of take-off: during the steady 30 A climb a low SoC and a high resistance look the same, and the sudden step to 15 A told them apart. Stage B learned $R_0 = 8.85\,\mathrm{m\Omega}$ (true 8.8, starting from 8.0). Its innovations looked like random noise inside their expected size, with a mean NIS of 0.51, slightly cautious by design of the 10 mV allowance.

5.2.2 Stress tests (E3–E7)

These runs used 10 averaged resting readings for the starting guess.

Sensor bias (E3). With a 0.8 A bias, charge counting ended 4.8% low, while stages B and C stayed within 2.5% (RMSE 1.28% and 1.58%). Stage C learned the bias only partly (+0.33 A against +0.8 A, with a wide band): one flight is not enough to separate it from the resistance, but the SoC stays right because the voltage keeps correcting it.

Aged battery (E4). This is the most important stress test (Figure 5.3). With a battery at 85% of its rated capacity, charge counting ended 10.2% too high and stage B 8.5% too high, its band holding the truth only 28% of the time: it blamed the lower voltage on its resistance instead of a smaller battery. Stage C ended only 0.2% off (RMSE 0.80%), with its capacity estimate falling from 5.0 to 4.45 Ah (true 4.25). Its bias estimate had the wrong sign, but together with the capacity it gave almost the right drain rate (3.17), which is what the SoC depends on.

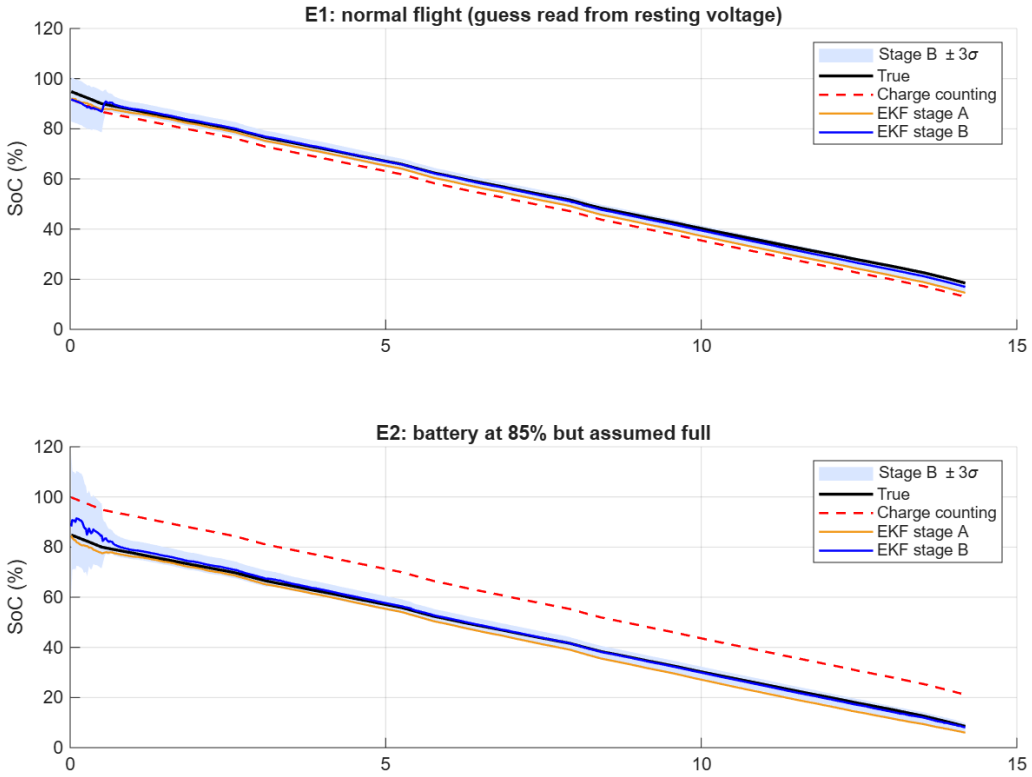


Figure 5.2: SoC tracking in E1 (top) and E2 (bottom). In E2 charge counting stays about 13% too high for the whole flight; stage B corrects the wrong start within a minute.

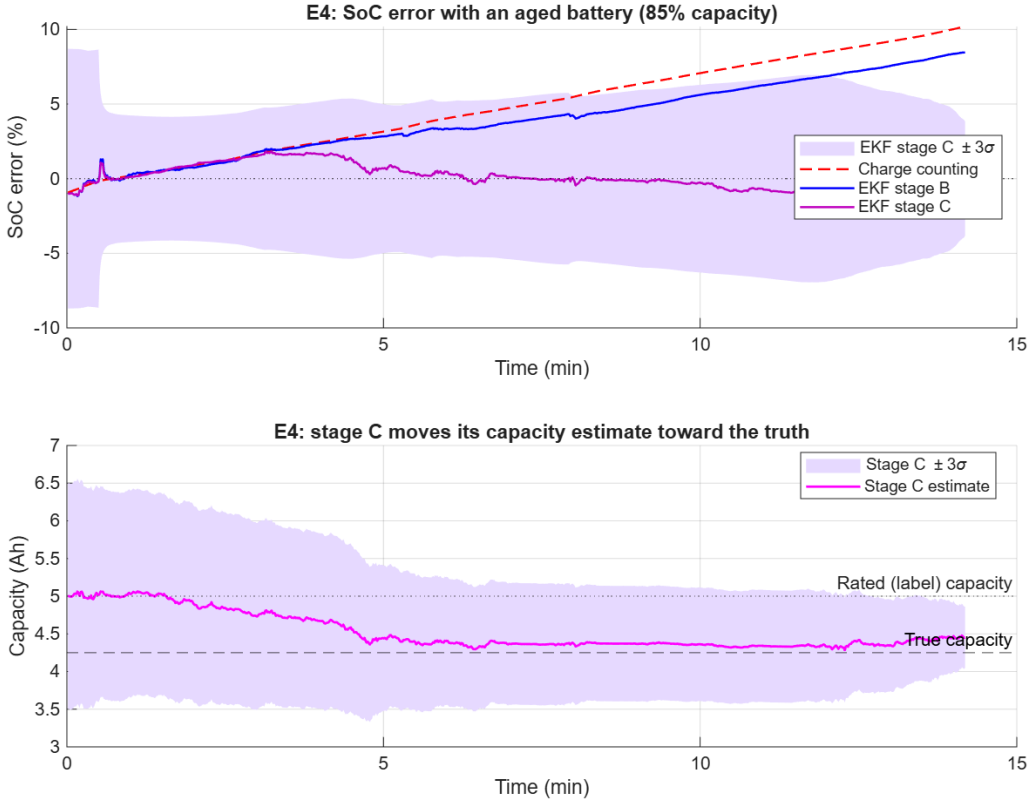


Figure 5.3: E4, aged battery. Top: SoC error; charge counting and stage B drift to about +8–10%, stage C stays near zero. Bottom: stage C's capacity estimate moves towards the true 4.25 Ah.

Tuning (E5). Telling the filter its voltage sensor was almost perfect ($R \times 0.01$) made it chase noise: it swung to -13% during take-off, was honest only 30% of the time, and its NIS rose to 32, a clear alarm. Telling it the sensor was nearly useless ($R \times 100$) kept it honest but with bands far too wide, which would make a drone turn back much too early. $Q \times 100$ made it jumpy (RMSE 2.35%). $Q \times 0.01$ was slightly more accurate (0.67% against 1.06%) but less honest (94%), so the chosen tuning was kept.

Strong mismatch (E6). With the stronger slow RC pair every filter degraded: stage A ended 7.9% low, stage C drifted to about -8% before recovering to -5%, and the best band (stage C) held the truth 75% of the time. Charge counting, which ignores the voltage, did about as well (RMSE 2.33%). This is the honest limit of the method: an EKF is only as good as its model.

Flat OCV region (E7). As the flight entered the flat part of the curve, where the slope falls from about 8 to 5 mV per 1% SoC, stage C's uncertainty rose from 1.5% to 2.2%. Around 65% SoC, where the slope has a small bump, the uncertainty dipped, following the slope closely. Stage B's uncertainty kept shrinking instead, because learning the resistance kept adding information.

5.2.3 Comparison over 20 flights (E8)

Table 5.2 averages 20 random flights per scenario. Stage C is the only filter honest in all four normal scenarios (E1–E4), with RMSE between 1.0% and 1.8%. Stage B is the most accurate on a healthy battery but fails on an aged one. Stage A is never honest, even though its NIS looks healthy (about 0.5): its normalised SoC error (NEES) was 30 to 400 times the expected value of 1, so its own signals cannot reveal its bias. These results confirm stage C as the filter for decisions.

Table 5.2: E8: mean SoC RMSE over 20 flights per scenario; in brackets, how often the truth was inside the filter's $\pm 3\sigma$ band.

Scenario	Charge counting	Stage A	Stage B	Stage C
E1 Nominal	1.83%	2.46% (16%)	1.03% (100%)	1.41% (100%)
E2 Wrong start	13.84%	2.47% (17%)	1.64% (100%)	1.80% (100%)
E3 Bias 0.8 A	2.64%	2.31% (22%)	1.27% (91%)	1.24% (100%)
E4 Aged battery	6.17%	1.39% (46%)	4.72% (27%)	1.03% (100%)
E6 Strong mismatch	1.83%	5.10% (6%)	2.37% (50%)	4.16% (83%)

5.3 Return-to-home decisions on one flight

The demonstration mission (aged battery, sensor reading 0.5 A low, 10% headwind) was flown with each policy (Table 5.3, Figure 5.4). Charge counting sat about 10% above the true SoC and kept drifting higher, so policy B pushed on to a second delivery, turned back at 8.2 min and died about 200 m from home. Policies A and C turned back at 6.4 min, right at the edge, and landed with just under 12%. Policy D turned back half a minute

earlier, when its estimate minus its uncertainty met the charge needed to get home, and landed with 17.8%.

Table 5.3: Demonstration mission: outcome under each policy.

Policy	Turn-back	Deliveries	True SoC at landing	Outcome
A. Voltage threshold	6.4 min	1	11.7%	Safe
B. Charge counting	8.2 min	2	0.0%	Battery died
C. EKF estimate only	6.4 min	1	11.9%	Safe
D. EKF risk-aware (95%)	5.9 min	1	17.8%	Safe

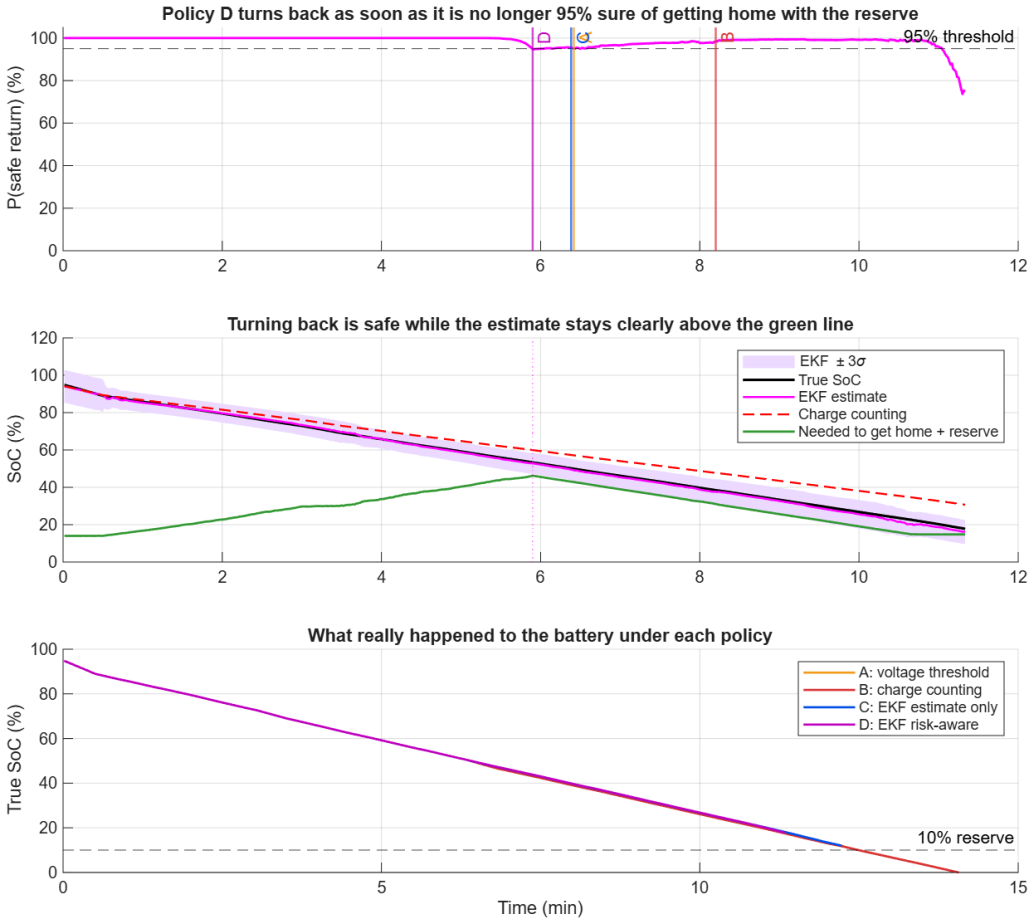


Figure 5.4: Demonstration mission. Top: probability of a safe return and each policy's turn-back time. Middle: EKF estimate with its band, charge counting, and the charge needed to get home plus the reserve (green). Bottom: true SoC under each policy; B runs out. (The dip in the top panel during landing is a display effect and affects no decision.)

5.4 Monte Carlo study

Table 5.4 gives the results of 500 random missions, flown by every policy under the same conditions and noise. Policy D was the only one with no deaths and a reserve-breach rate (2.8%) inside its 5% promise. Policy C, with the same filter but ignoring its uncertainty,

aims exactly at the reserve and broke it in 38% of missions. Policy B was dangerous: it died in 30% of all missions and 58% of those with an aged battery. Policy A died in 9% of missions and still left 8% of the battery unused. Over these missions the stage-C EKF tracked the SoC with an RMSE of 1.31%, against 3.35% for charge counting, and its band held the truth 100% of the time.

Table 5.4: Monte Carlo results over 500 missions. Reserve broken: died, or landed below the 10% reserve. Unused: mean SoC left above the reserve at landing (negative means below it). Deliveries count a drop once its hover is finished.

Policy	Died	Reserve broken	Unused	Deliveries	Died, aged only
A. Voltage (3.5 V)	9.2%	32.2%	8.0%	0.85	10.0%
B. Charge counting	30.4%	90.2%	−4.1%	1.56	58.2%
C. EKF estimate	0.0%	38.0%	0.6%	1.32	0.0%
D. EKF risk-aware (95%)	0.0%	2.8%	6.1%	1.16	0.0%

5.5 Safety versus wasted battery

Figure 5.5 repeats 200 of the missions with different thresholds. Policy D kept its promise at every setting: at 80%, 90%, 95% and 99% it broke the reserve in 9.5%, 4.0%, 2.0% and 0% of missions, each within the promised 20%, 10%, 5% and 1%, so its probabilities can be trusted. (With 200 missions, the 0.1% promise at 99.9% can be neither confirmed nor contradicted.) Going from 90% to 99% safety cost 3.2% more unused battery. No voltage threshold was both safe and efficient: 3.40 V killed 96% of drones, while 3.55 V, the setting that matches D's safety, wasted 17.9% of the battery and completed half as many deliveries. For every reasonably safe voltage threshold, a setting of D was at least as safe and wasted less.

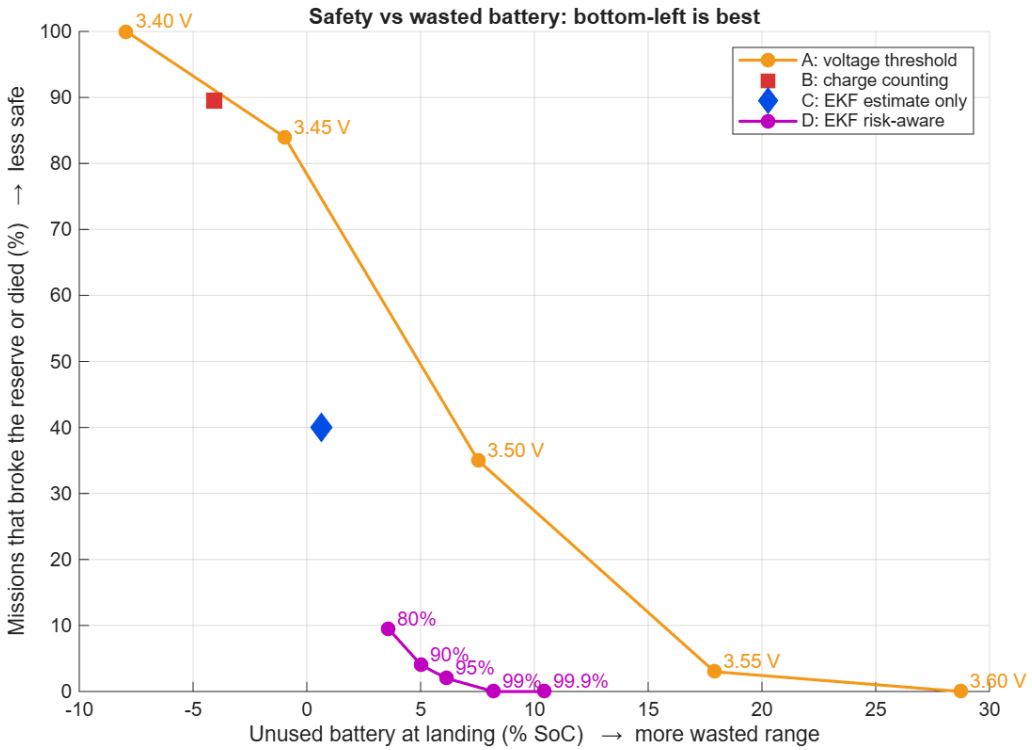


Figure 5.5: Safety against wasted battery over 200 missions (bottom-left is best). Each point of A is a voltage threshold, each point of D a probability threshold.

Chapter 6

Discussion

6.1 Why the filter works, and where it is unsure

Charge counting has no way to notice its own mistakes, so a wrong start, a sensor bias or a smaller battery stay wrong for the whole flight. The EKF gets an independent check from the voltage every second, and the Kalman gain decides how much to trust it (Section 2.2). That check is weakest where the OCV curve is flat: there a voltage error looks like a large SoC error, so the gain is small and the uncertainty grows. Experiment E7 showed exactly this, with stage C's uncertainty rising as the slope fell and dipping where the slope briefly rose. The filter knows when it knows less, which is what the return-to-home rule needs.

6.2 What one flight can and cannot reveal

The resistance shows itself whenever the current changes, because only the sag changes with the current while the OCV does not; in E1 and E2 the end of take-off was enough to learn it. The bias and the capacity are different. Within one flight of mostly steady cruise their effects on the voltage overlap with each other and with the resistance (3.16), so they cannot be told apart, and stage C's individual estimates were sometimes wrong, even in sign (E3, E4). What the voltage does fix is their combination, the drain rate (3.17), and that is all the SoC and the trip cost need. This is why the trip cost uses the rate and not the capacity alone.

6.3 Model errors and tuning

The largest errors came from the model, not from the sensors. A resistance that is 10% wrong makes the filter consistently misread the voltage; this is not random noise, so telling the filter its sensor is noisier does not remove it (Section 3.4.5). Worse, the filter cannot see the problem itself: stage A's innovations looked healthy (NIS about 0.5) while its error, checked against the truth, was 30 to 400 times larger than it claimed (E8). Only

a simulation, where the truth is known, exposes this. Learning the resistance removed most of the error; the strong mismatch of E6 shows that a large enough model error defeats every version of the filter.

Tuning sets how cautious the filter is rather than how accurate. A too-small R made it chase noise, and its NIS of 32 raised the alarm; a too-large R kept it honest but would make the drone turn back far too early (E5). The chosen settings are slightly cautious on purpose, because the decision rule turns the filter's uncertainty directly into a safety margin.

A more advanced filter would not fix the main problem. With a perfect model the EKF's error was only 0.12%, so the tangent-line approximation is not what limits accuracy; the model is. An unscented Kalman filter or a particle filter [3, 10] would cost more computation and still inherit the same model error.

6.4 The most accurate filter is not the safest

Stage B was the most accurate filter on a healthy battery, yet on an aged battery it was confidently wrong, and wrong in the dangerous direction: it believed there was more charge than there was (E4, E8). Stage C was slightly less accurate on a healthy battery but honest in every normal scenario. For a decision that depends on the uncertainty, honesty matters more than the last fraction of a percent of accuracy, so stage C drives the return-to-home rule.

The Monte Carlo study shows the value of using that uncertainty. Policies C and D use the same filter and differ only in whether they use its uncertainty. Policy C aims exactly at the reserve, so it lands below it about as often as above, breaking it in 38% of missions. Policy D keeps a buffer that grows when the filter is unsure, and broke the reserve in 2.8% of missions, inside its 5% promise (Table 5.4). At every threshold the share of missions that broke the reserve stayed within the promise (Figure 5.5), so the probabilities mean what they say. This tests the whole chain at once: the model, the filter's uncertainty, the trip cost and the probability. The price is modest: about 6% of the battery unused and slightly fewer deliveries than policy C, and an operator can choose a different threshold to move along the trade-off.

The baselines fail for clear reasons. Charge counting becomes optimistic with an aged battery or a sensor that reads low, and died in 58% of aged-battery missions. A voltage threshold ignores how far the drone is from home and is fooled by sag, so no single threshold is both safe and efficient.

6.5 Limitations and future work

- **Simulation only.** The results come from a simulated battery and drone; real flight logs, such as the public datasets [11, 13], do not contain the true SoC, but could test the voltage predictions.

- **Representative battery data.** The OCV curve is a typical lithium-ion shape, not measured on a specific cell; temperature and voltage hysteresis are not modelled.
- **Simple decision model.** The rule assumes bell-shaped errors, a straight flight home and a constant cruise current, and does not add the uncertainty of the drain rate itself.
- **One flight is not enough** to learn the bias and capacity separately. Calibrating the current sensor at rest before take-off, when the true current is zero, and estimating the capacity between flights from the charge used and the resting voltage before and after, would fix both cheaply.
- **Model error.** A two-RC filter model, or noise settings that adapt during flight, could reduce the errors seen under strong mismatch (E6).

Chapter 7

Conclusion

This project let a delivery drone decide when to fly home using an Extended Kalman Filter that estimates the battery's state of charge and how uncertain it is. All four objectives were met. The MATLAB simulator reproduced realistic battery behaviour, including a voltage sag of 0.37 V per cell at take-off, and its battery deliberately differs from the filter's model. Learning the internal resistance during flight made the EKF accurate and honest: on a normal flight its SoC error was 0.97% against 4.36% for charge counting, and it corrected a 15% wrong start within 42 s while charge counting stayed wrong all flight. The full filter, which also learns the sensor bias and capacity, was the only one honest in every normal scenario, including an aged battery on which the most accurate filter overestimated the charge by 8.5%. The risk-aware return-to-home rule built on it brought the drone home in all 500 random missions and broke the 10% reserve in only 2.8% of them, within its 5% promise, while charge counting let 30% of drones run out and the same filter used without its uncertainty broke the reserve in 38% of missions. At every threshold tested the rule kept its promise, so its probabilities can be trusted.

The work is limited to simulation and a representative battery curve; the most useful next steps are calibrating the sensor bias before take-off, estimating the capacity between flights and testing on real flight data (Section 6.5).

The main lesson is that, for a safety decision, a filter's uncertainty is as valuable as its estimate, provided that uncertainty is honest.

References

[1] G. L. Plett, "Extended Kalman filtering for battery management systems of LiPB-based HEV battery packs: Part 1. Background," *Journal of Power Sources*, vol. 134, no. 2, pp. 252–261, 2004. [Online]. Available: https://doi.org/10.1016/j.jpowsour.2004.02.031

[2] ——, "Extended Kalman filtering for battery management systems of LiPB-based HEV battery packs: Part 3. State and parameter estimation," *Journal of Power Sources*, vol. 134, no. 2, pp. 277–292, 2004. [Online]. Available: https://doi.org/10.1016/j.jpowsour.2004.02.033

[3] D. Simon, *Optimal State Estimation: Kalman, H_∞, and Nonlinear Approaches*. Hoboken, NJ: Wiley-Interscience, 2006.

[4] G. Welch and G. Bishop, "An introduction to the Kalman filter," Department of Computer Science, University of North Carolina at Chapel Hill, Tech. Rep. TR 95-041, 2004, updated 5 April 2004. [Online]. Available: https://techreports.cs.unc.edu/papers/95-041.pdf

[5] G. L. Plett, "Extended Kalman filtering for battery management systems of LiPB-based HEV battery packs: Part 2. Modeling and identification," *Journal of Power Sources*, vol. 134, no. 2, pp. 262–276, 2004. [Online]. Available: https://doi.org/10.1016/j.jpowsour.2004.02.032

[6] ——, *Battery Management Systems, Volume II: Equivalent-Circuit Methods*. Artech House, 2015.

[7] E. Siampis, "Extended Kalman filter with battery estimation example," MATLAB Central File Exchange, version 1.0.0, 2022, accessed 19 Oct. 2026. [Online]. Available: https://www.mathworks.com/matlabcentral/fileexchange/105855-extended-kalman-filter-with-battery-estimation-example

[8] R. Schacht-Rodríguez, G. Ortiz-Torres, C. D. García-Beltrán, C. Astorga-Zaragoza, J.-C. Ponsart, and D. Theilliol, "SoC estimation using an extended Kalman filter for UAV applications," in *Proc. International Conference on Unmanned Aircraft Systems (ICUAS)*, Miami, FL, USA, 2017, pp. 179–187. [Online]. Available: https://hal.archives-ouvertes.fr/hal-01537730

[9] E. F. Hogge, B. M. Bole, S. L. Vazquez, C. S. Kulkarni, T. H. Strom, B. L. Hill, K. M. Smalling, and C. C. Quach, "Verification of prognostic algorithms to predict remaining flying time for electric unmanned vehicles," *International Journal of Prognostics and Health Management*, vol. 9, no. 1, 2018, accepted manuscript

also available as NASA Technical Reports Server document 20190025952. [Online]. Available: https://papers.phmsociety.org/index.php/ijphm/article/view/2700

[10] H. Dibbern, M. Roßberg, and C. Werner, "Comparative evaluation of investigation methods for estimating the load-dependent state of charge and end of discharge of a multirotor UAV battery," *Journal of Aviation Technology and Engineering*, vol. 12, no. 2, p. Article 4, 2023. [Online]. Available: https://docs.lib.purdue.edu/jate/vol12/iss2/4

[11] T. A. Rodrigues, J. Patrikar, A. Choudhry, J. Feldgoise, V. Arcot, A. Gahlaut, S. Lau, B. Moon, B. Wagner, H. S. Matthews, S. Scherer, and C. Samaras, "In-flight positional and energy use data set of a DJI Matrice 100 quadcopter for small package delivery," *Scientific Data*, vol. 8, p. Article 155, 2021. [Online]. Available: https://doi.org/10.1038/s41597-021-00930-x

[12] A. Bills, V. Viswanathan, S. Sripad, E. Frank, D. Charles, and W. L. Fredericks, "eVTOL battery dataset," Carnegie Mellon University, KiltHub repository, version 3, 2021. [Online]. Available: https://doi.org/10.1184/R1/14226830.v3

[13] A. Bills, S. Sripad, L. Fredericks, M. Guttenberg, D. Charles, E. Frank, and V. Viswanathan, "A battery dataset for electric vertical takeoff and landing aircraft," *Scientific Data*, vol. 10, p. Article 344, 2023. [Online]. Available: https://doi.org/10.1038/s41597-023-02180-5

[14] P. Kollmeyer, M. Naguib, and M. Skells, "LG 18650HG2 li-ion battery data," Mendeley Data, McMaster University, version 2, 2020. [Online]. Available: https://doi.org/10.17632/b5mj79w5w9.2

Appendix A

MATLAB Code

This appendix lists the core files of the method: the true battery and sensors, the EKF, and the return-to-home decision. The remaining files (parameters, OCV lookup, drone motion, performance measures, experiment and plotting scripts) are in the project's code folder `drone_battery_ekf/`, organised as in Table 4.1.

A.1 Simulated battery and sensors

```
function [b, V] = battery_true_step(b, I, pt, dt)
% BATTERY_TRUE_STEP  The "real" battery: a 2-RC model, one time step.
%
%   b  : battery state (soc, V1, V2)
%   I  : true current drawn during this step (A, positive = discharge)
%   pt : true parameters (p.true from battery_params)
%   V  : terminal voltage of one cell at the END of the step (V)
%
% This model is deliberately richer than the filter's 1-RC model.

a1 = exp(-dt / (pt.R1 * pt.C1));
a2 = exp(-dt / (pt.R2 * pt.C2));

b.soc = b.soc - I * dt / (3600 * pt.Cn);
b.V1  = a1 * b.V1 + pt.R1 * (1 - a1) * I;
b.V2  = a2 * b.V2 + pt.R2 * (1 - a2) * I;

V = ocv_lookup(b.soc, pt.ocv_offset) - pt.R0 * I - b.V1 - b.V2;
end
```

Listing A.1: `sim/battery_true_step.m`

```
function [Im, Vm] = read_sensors(I, V, ps)
% READ_SENSORS  What the drone's sensors report (imperfect).
%
%   I, V : true current (A) and true cell voltage (V)
%   ps   : sensor settings (p.sens from battery_params)
%   Im   : measured current = true + constant bias + noise
%   Vm   : measured cell voltage = true + noise

```

```
Im = I + ps.bias + ps.sigI * randn;
Vm = V + ps.sigV * randn;
end
```

Listing A.2: `sim/read_sensors.m`

A.2 Extended Kalman Filter

```
function f = ekf_init(p, stage, soc0, sig_soc0)
% EKF_INIT  Set up the filter: starting guess, starting uncertainty, tuning.
%
%   f = ekf_init(p, stage, soc0, sig_soc0)
%
%   p        : parameters from battery_params (the filter uses p.filt only)
%   stage    : which states the filter is allowed to learn
%              'A' -> SoC, V1                          (basic EKF)
%              'B' -> SoC, V1, R0                      (main filter: also learns resistance)
%              'C' -> SoC, V1, R0, bias b, capacity Cn (everything)
%   soc0     : starting SoC guess (fraction)
%   sig_soc0 : how unsure we are about that guess (1 sigma, fraction)
%
% The state is always x = [SoC; V1; b; Cn; R0]. States a stage does not
% learn are frozen at their nominal values (zero uncertainty and zero
% process noise), which is the same as leaving them out of the filter.

f.stage = upper(stage);

% What the filter believes about the battery
f.R1 = p.filt.R1;   f.C1 = p.filt.C1;
f.ocv_offset = p.filt.ocv_offset;

% Noise settings
f.sigI = p.sens.sigI;                                % current sensor noise (A)
f.Rv   = p.sens.sigV^2 ...                           % voltage sensor noise
       + (p.filt.R0 * p.sens.sigI)^2 ...             % current noise seen through R0
       + p.filt.sig_model^2;                         % allowance for model error (V^2)

% Starting guess and uncertainty (1 sigma) for [SoC, V1, b, Cn, R0]
x0  = [soc0; 0; 0; p.filt.Cn; p.filt.R0];
sd0 = [sig_soc0, 0.01, 1.0, 0.5, 0.004];             % SoC, 10 mV, 1 A, 0.5 Ah, 4 mOhm

% Process noise per step: how much each state may drift in ways the
% model does not explain (variances)
q = [1e-8, 1e-6, 1e-6, 1e-7, 1e-9];

switch f.stage
    case 'A', learn = [1 1 0 0 0];
    case 'B', learn = [1 1 0 0 1];
    case 'C', learn = [1 1 1 1 1];
    otherwise, error('Stage must be ''A'', ''B'' or ''C''.');
end
```

```
sd0(~learn) = 0;
q(~learn)   = 0;

f.learn = logical(learn);
f.x = x0;
f.P = diag(sd0.^2);
f.q = q;
end
```

Listing A.3: `estimators/ekf_init.m`

```
function [f, info] = ekf_step(f, Im, Vm, dt)
% EKF_STEP  One predict + update of the battery EKF.
%
%   f    : filter (from ekf_init), holds the estimate f.x and uncertainty f.P
%   Im   : measured current during this step (A)
%   Vm   : measured cell voltage at the end of the step (V)
%   info : innovation, its expected size, gain, ... (for plots and checks)
%
% State x = [SoC; V1; b; Cn; R0].

soc = f.x(1);  v1 = f.x(2);  b = f.x(3);  Cn = f.x(4);  R0 = f.x(5);
I = Im - b;                                     % bias-corrected current
a = exp(-dt / (f.R1 * f.C1));                   % how much of V1 survives one step

% ---------- 1. Predict: move the estimate with the model ----------
x_pred = [soc - I*dt/(3600*Cn);                 % charge counting
          a*v1 + f.R1*(1 - a)*I;                % slow voltage relaxes toward R1*I
          b;                                    % bias, capacity and resistance
          Cn;                                   %   stay the same (apart from
          R0];                                  %   slow random wander)

F = eye(5);                                     % Jacobian of the process model
F(1,3) = dt/(3600*Cn);                          %   dSoC/db
F(1,4) = I*dt/(3600*Cn^2);                      %   dSoC/dCn
F(2,2) = a;                                     %   dV1/dV1
F(2,3) = -f.R1*(1 - a);                         %   dV1/db

G = [-dt/(3600*Cn); f.R1*(1 - a); 0; 0; 0];     % how current-sensor noise enters
Q = G * f.sigI^2 * G' + diag(f.q);
P_pred = F * f.P * F' + Q;                      % uncertainty grows

% ---------- 2. Compare: expected vs measured voltage ----------
[ocv, docv] = ocv_lookup(x_pred(1), f.ocv_offset);
I_pred = Im - x_pred(3);
V_pred = ocv - x_pred(2) - x_pred(5)*I_pred;
y = Vm - V_pred;                                % innovation (the surprise)

H = [docv, -1, x_pred(5), 0, -I_pred];          % Jacobian of the measurement model
S = H * P_pred * H' + f.Rv;                     % expected size of the surprise (one
    number)

% ---------- 3. Update: move by K x surprise ----------
K = P_pred * H' / S;                            % Kalman gain (5x1)
x = x_pred + K * y;
IKH = eye(5) - K*H;
P = IKH * P_pred * IKH' + K * f.Rv * K';        % Joseph form: stays symmetric and
   positive
```

```
P = (P + P') / 2;

x(1) = min(max(x(1), 0), 1);                    % SoC stays in [0, 1]
x(4) = max(x(4), 0.5);                          % capacity stays positive
x(5) = max(x(5), 1e-4);                         % resistance stays positive

f.x = x;  f.P = P;
info.y = y;  info.S = S;  info.K = K;  info.V_pred = V_pred;
info.nis = y^2 / S;                             % normalised innovation squared
end
```

Listing A.4: `estimators/ekf_step.m`

A.3 Return-to-home decision

```
function dsoc = trip_soc(dist_home, I_cruise, b, Cn, m)
% TRIP_SOC  Charge (as a fraction of SoC) needed to fly home and land.
%
%   dist_home : straight-line distance to home (m)
%   I_cruise  : cruise current as the SENSOR reads it (A), learned in flight
%   b         : estimated current-sensor bias (A); 0 if unknown
%   Cn        : capacity to divide by (Ah)
%   m         : mission (speed, landing time and current)
%
% The cruise part uses the same drain rate the filter uses, (I - b)/Cn,
% because that combination is what the voltage makes observable (Day 4).
% I_cruise is a MEASURED current, so the bias is removed from it; the
% landing current m.I_land is a nominal TRUE current, so it is used as is.

t_cruise = dist_home / m.v_cruise;                      % s
charge   = (I_cruise - b) * t_cruise + m.I_land * m.T_land;   % A*s
dsoc     = charge / (3600 * Cn);
end
```

Listing A.5: `decision/trip_soc.m`

```
function [ps, margin, dsoc, sig_total] = p_safe_return(f, dist_home, I_cruise, m,
    rp)
% P_SAFE_RETURN  Probability of getting home with the reserve still left.
%
%   f         : EKF (stage C) with estimate f.x and uncertainty f.P
%   dist_home : distance to home (m)
%   I_cruise  : learned cruise current as measured (A)
%   m         : mission
%   rp        : RTH settings: rp.reserve (fraction), rp.trip_frac (fraction)
%
%   ps        : P(safe return) = Phi(margin / sig_total)
%   margin    : SoC estimate - charge needed - reserve
%   dsoc      : charge needed to get home (fraction)
%   sig_total : combined uncertainty of SoC and of the trip (fraction)

soc = f.x(1);  b = f.x(3);  Cn = f.x(4);
dsoc   = trip_soc(dist_home, I_cruise, b, Cn, m);
margin = soc - dsoc - rp.reserve;

```

```
sig_soc   = sqrt(f.P(1,1));          % from the EKF: how unsure is the SoC
sig_trip  = rp.trip_frac * dsoc;     % wind and load surprises on the way home
sig_total = sqrt(sig_soc^2 + sig_trip^2);

ps = 0.5 * erfc(-margin / (sqrt(2) * sig_total));   % normal CDF, base MATLAB
end
```

Listing A.6: `decision/p_safe_return.m`

```
function [go_home, low_count] = rth_policy(policy, s, rp)
% RTH_POLICY  The four turn-back rules. Returns true when the drone should go home.
%
%   policy : 'A' voltage threshold
%            'B' charge counting + margin rule
%            'C' EKF estimate + margin rule (ignores uncertainty)
%            'D' EKF risk-aware: P(safe return) below a threshold
%   s      : what the drone knows this second (fields used below)
%   rp     : settings (rp.V_thr, rp.V_hold, rp.reserve, rp.p_thr)

low_count = s.low_count;
switch policy
    case 'A'   % voltage stays below the threshold for V_hold seconds
        if s.Vm < rp.V_thr, low_count = low_count + 1; else, low_count = 0; end
        go_home = low_count >= rp.V_hold;
    case 'B'   % charge counting says the margin is gone
        go_home = s.soc_cc - s.dsoc_cc - rp.reserve < 0;
    case 'C'   % EKF estimate says the margin is gone
        go_home = s.margin < 0;
    case 'D'   % EKF is no longer sure enough of getting home with the reserve
        go_home = s.ps < rp.p_thr;
    otherwise
        error('Unknown policy "%s".', policy);
end
end
```

Listing A.7: `decision/rth_policy.m`

```
function out = run_mission_rth(p, m, c, policy, rp, seed)
% RUN_MISSION_RTH  Fly a mission that is too long for one battery and let a
% policy decide when to turn back.
%
%   p      : battery_params()
%   m      : mission (use make_mission('long'))
%   c      : random conditions from draw_conditions (start SoC, bias, wind, age ...)
%   policy : 'A', 'B', 'C' or 'D' (see rth_policy)
%   rp     : RTH settings (reserve, trip_frac, p_thr, V_thr, V_hold)
%   seed   : random seed for the sensor noise
%
%   out.died      : true if the battery died in the air
%   out.soc_land  : true SoC at landing
%   out.drops     : deliveries completed
%   out.t_rth     : second at which the policy turned back (NaN = never)
%   out.rec       : second-by-second record (for plots)
%
% The stage C EKF and charge counting always run, whichever policy decides.

```

```
% ---- Apply this mission's conditions ----
p.sens.bias = c.bias;
p.true.Cn = c.Cn;  p.true.R0 = c.R0;  p.true.R1 = c.R1;  p.true.C1 = c.C1;  p.true.R2 = c.R2;
m.wind = c.wind;
dt = p.dt;
rng(seed);

% ---- Before takeoff: average 10 resting readings for the starting guess ----
batt = battery_true_init(c.soc0);
V_rest = ocv_lookup(c.soc0, p.true.ocv_offset);
Vm0 = 0;
for i = 1:10
    [~, Vm_i] = read_sensors(0, V_rest, p.sens);
    Vm0 = Vm0 + Vm_i / 10;
end
soc_guess0 = soc_from_ocv(Vm0, p.filt.ocv_offset);

f      = ekf_init(p, 'C', soc_guess0, 0.03);     % stage C drives the decision
soc_cc = soc_guess0;                             % charge counting starts from the same guess
drone  = drone_init(m);
I_cr   = m.I_cruise;                             % learned cruise current (as measured)

% ---- Storage ----
N = 4000;
rec.t = nan(1,N);  rec.soc = nan(1,N);  rec.soc_hat = nan(1,N);  rec.sig = nan(1,N);
rec.soc_cc = nan(1,N);  rec.ps = nan(1,N);  rec.margin = nan(1,N);  rec.dsoc = nan(1,N);
rec.Vm = nan(1,N);  rec.pos = nan(N,2);  rec.mode = strings(1,N);

wobble = 0;  low_count = 0;
out.died = false;  out.t_rth = NaN;
for k = 1:N
    [drone, I_demand] = drone_step(drone, m, dt);
    if I_demand > 0
        wobble = 0.9*wobble + 0.3*randn;
        I = max(I_demand + wobble, 0);
    else
        I = 0;
    end

    [batt, V] = battery_true_step(batt, I, p.true, dt);
    [Im, Vm]  = read_sensors(I, V, p.sens);
    f         = ekf_step(f, Im, Vm, dt);
    soc_cc    = coulomb_count(soc_cc, Im, p.filt.Cn, dt);
    if strcmp(drone.mode, 'cruise')
        I_cr = 0.98*I_cr + 0.02*Im;              % slowly learn the real cruise current
    end

    % ---- What the drone knows, and the decision ----
    dist_home = norm(drone.pos - m.home);
    [ps, margin, dsoc] = p_safe_return(f, dist_home, I_cr, m, rp);
    if any(strcmp(drone.mode, {'cruise', 'hover'}))       % decide only while working
```

```
        s.Vm = Vm;  s.low_count = low_count;  s.ps = ps;  s.margin = margin;
        s.soc_cc  = soc_cc;
        s.dsoc_cc = trip_soc(dist_home, I_cr, 0, p.filt.Cn, m);   % no bias, label capacity
        [go_home, low_count] = rth_policy(policy, s, rp);
        if go_home
            drone.mode = 'return';  drone.t_in_mode = 0;
            out.t_rth = k;
        end
    end

    rec.t(k) = k*dt;  rec.soc(k) = batt.soc;  rec.soc_hat(k) = f.x(1);  rec.sig(k) = sqrt(f.P(1,1));
    rec.soc_cc(k) = soc_cc;  rec.ps(k) = ps;  rec.margin(k) = margin;  rec.dsoc(k) = dsoc;
    rec.Vm(k) = Vm;  rec.pos(k,:) = drone.pos;  rec.mode(k) = drone.mode;

    if I > 0 && (V < p.true.V_cutoff || batt.soc <= 0)
        out.died = true;  break;                % battery died in the air
    end
    if strcmp(drone.mode, 'landed'), break; end
end

% ---- Trim and summarise ----
n = k;
names = fieldnames(rec);
for i = 1:numel(names)
    v = rec.(names{i});
    if size(v, 2) == N, rec.(names{i}) = v(:, 1:n); else, rec.(names{i}) = v(1:n, :); end
end
out.soc_land  = batt.soc;
out.drops     = drone.wp_index - 1;          % a drop counts once its hover is finished
e_ekf = rec.soc_hat - rec.soc;               % how well the estimators tracked SoC
e_cc  = rec.soc_cc  - rec.soc;
out.rmse_ekf  = sqrt(mean(e_ekf.^2));
out.rmse_cc   = sqrt(mean(e_cc.^2));
out.inside    = mean(abs(e_ekf) <= 3*rec.sig);
out.dist      = drone.dist_flown;
out.T         = n * dt;
out.rec       = rec;
end
```

Listing A.8: `sim/run_mission_rth.m`